Esercitazione 8 di Laboratorio Informatica@DSS (2023/2024)

Paolo Franciosa

None

Lo scopo del laboratorio è di esercitarsi e misurare la propria preparazione: gli esercizi non sono troppo difficili, se si sono seguite le lezioni. Non vi viene comunque messo alcun voto.

Modalità di lavoro: gli studenti devono lavorare in autonomia o in piccoli gruppi, senza disturbare i colleghi. Il lavoro di gruppo è fruttuoso solo se tutti partecipano e se ognuno scrive una propria soluzione per tutti gli esercizi.

Il docente cercherà per quanto possibile di non occupare il tempo del laboratorio per introdurre materiale nuovo, anche se a volte questo sarà necessario. Il docente è a disposizione per aiutare gli studenti, che possono iniziare a lavorare anche prima che il docente arrivi in aula, se lo desiderano

Raccomandazioni: leggete bene il testo degli esercizi prima di chiedere chiarimenti. In ogni caso sarò in aula con voi.

Uso dei file di test: per aiutarvi a completare questa esercitazione avete a disposizione dei programmi di test per testare la vostra soluzione. Questi sono simili a quelli che avrete in sede di esame, pertanto vi consiglio di imparate ad usarli. Per portare a termine l'esercizio è necessario

- scrivere un file di soluzione **col nome specificato**;
- avere dentro la funzione **col nome specificato**;
- porre il file di test corrispondente **nella stessa cartella**;
- eseguire in quella cartella il comando `python3 <fileditest>`

dove `<fileditest>` va ovviamente sostituito con il nome del file di test appropriato per l'esercizio su cui state lavorando. Per ogni esercizio ci sta un file di test indipendente, così da poter lavorare sugli esercizi uno alla volta con più agio.

Il risultato di ogni test è una schermata (o più schermate) nella quale si mostra:

- se è stato trovato il file con il nome corretto,
- se il file contiene la funzione con il nome corretto,
- se chiamate alla funzione con diversi valori dei parametri terminano restituendo risultati corretti.

Per ogni funzione scritta vengono eseguite chiamate con diversi valori dei parametri. L'esito dei test viene riportato con il carattere

- `E` se la chiamata non può essere eseguita,
- `F` se la funzione non restituisce il risultato corretto,
- `.` se la funzione restituisce il risultato corretto.

1 Conta le righe di un file di testo

Scrivere una funzione `conta_righe_file(nomefile)` che riceva come argomento il nome di un file di testo e restituisca il numero di righe del file. Ad esempio, per il file `conta_righe_file1.txt` contenente

```
1;3;25
10;12
43
7;2;-6
```

deve essee restituito il valore:

```
conta_righe_file('conta_righe_file1.txt')
```

```
4
```

Il programma Python deve essere salvato nel file: `conta_righe_file.py`

File di test: `test_conta_righe_file.py`

2 Conta le righe non vuote di un file di testo

Scrivere una funzione `conta_righe_nonvuote_file(nomefile)` che riceva come argomento il nome di un file di testo e restituisca il numero di righe del file che contengono caratteri diversi da `whitespace`. Il metodo `.strip()` applicato su un oggetto `str` restituisce una copia della stringa eliminando tutte le occorrenze dei cosiddetti "whitespace" in testa e in coda alla stringa, cioè spazi, newline e tab.

Ad esempio, se `s = "  abc  de \t  \n"`

allora `s.strip()` restituisce la stringa `"abc  de"`

Quindi, la funzione applicata al file contenente

```
3

4;1

-2;6
-5
```

deve restituire

```
conta_righe_nonvuote_file('conta_righe_nonvuote_file3.txt')
```

```
4
```

Il programma Python deve essere salvato nel file: `conta_righe_nonvuote_file.py`

File di test: `test_conta_righe_nonvuote_file.py`

3 Carica dizionario da file

Scrivere una funzione `carica_dizionario(nomefile)` che riceva come argomento il nome di un file di testo, e che carichi da quel file di testo il contenuto di un dizionario. La funzione si aspetta un file contenente una serie di righe, ciascuna delle quali contiene una coppia `chiave valore`, separati da spazi, dove `valore` è un numero intero.

Ad esempio, il file `carica_dizionario1.txt` contenente

```
Gianni 23
Dario 34
Anna 23
```

rappresenta il dizionario

```
{'Gianni': 23, 'Anna': 23, 'Dario': 34}
```

Quindi:

```
carica_dizionario('carica_dizionario1.txt')
```

```
{'Gianni': 23, 'Dario': 34, 'Anna': 23}
```

Suggeriamo, una volta ottenuto il testo contenuto in ciascuna riga sotto forma di stringa, di usare semplicemente il metodo `.split()` per estrarre i dati.

Il programma deve sollevare l'errore `FileNotFoundError` se il file non esiste.

Il programma Python deve essere salvato nel file: `carica_dizionario.py`

File di test: `test_carica_dizionario.py`

4 Ricerca pattern in un file

Scrivere una funzione `ricerca_linee(filepath, pattern)` che restituisce una lista di tutte le righe nel file di testo `filepath` che contengono come sottostringa la stringa passata come secondo parametro. Ad esempio, se la stringa è contenuta solo nella prima e nella terza riga, si deve restituire la lista [1, 3]. Contrariamente a quando accade generalmente in Python, questa volta la posizione delle righe parte da 1.

Ad esempio, se il parametro `pattern` vale `"ato"` e il contenuto del file è il seguente

```
Il cognato del
nipote non si è
presentato
all'appuntamento di ieri con il senatore
```

la funzione deve restituire la lista [1, 3, 4]

Per verificare se la stringa `s1` contiene la stringa `s2` basta usare il predicato `s2 in s1`

```
ricerca_linee('ricerca_linee1.txt', "ato")
```

```
[1, 3, 4]
```

Il programma Python deve essere salvato nel file: `ricerca_linee.py`

File di test: `test_ricerca_linee.py`

5 Somma interi da file di testo

Scrivere una funzione `lista_somme_da_file(nomefile)` che riceva come argomento il nome di un file di testo, che contiene interi separati da `;` su ciascuna riga, e restituisca la lista delle somme dei valori contenuti in ciascuna riga. La funzione deve interpretare il contenuto del file come una serie di numeri **interi**.

Ad esempio, per il file `lista_somme_da_file1.txt` contenente

```
1;3;25
10;12
43
7;2;-6
```

deve essere restituita la lista:

```
lista_somme_da_file('lista_somme_da_file1.txt')
```

```
[29, 22, 43, 3]
```

Importante: Deve essere letta una riga alla volta

Suggeriamo, una volta ottenuto il testo in una generica riga del file in forma di stringa, di usare semplicemente il metodo `.split(";")` per estrarre i dati separati da `;`

Il programma Python deve essere salvato nel file: `lista_somme_da_file.py`

File di test: `test_lista_somme_da_file.py`